spinalSynth

Table of Contents

Additional documents for spinalSynth:

- Implementation_specs.md
- Testing_specs.md

1. Introduction

This project implements a compact digital audio synthsizer in SpinalHDL.

The oscillator is based on Direct Digital Synthesis (DDS) using a phase accumulator architecture. The oscillator generates audio waveforms internally using an oversampled DDS engine and outputs stereo audio using the I²S protocol.

The project is intentionally designed to remain:

- compact
- deterministic
- FPGA-friendly
- easy to understand
- easy to simulate
- easy to extend later

Features

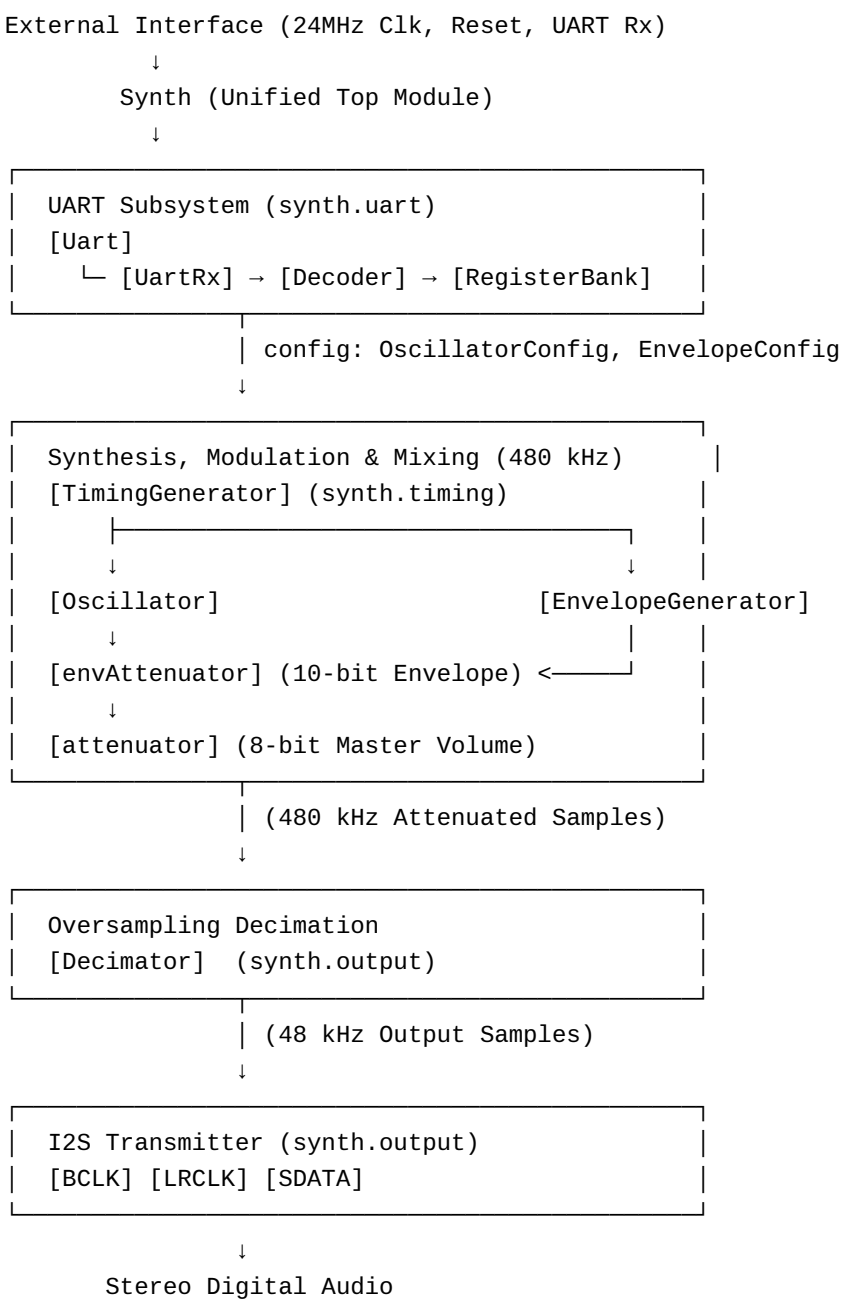
- 24-bit DDS phase accumulator
- 480 kHz internal DDS update rate
- 48 kHz stereo audio output
- 16-bit signed audio samples
- Stereo I²S output interface
- Oversampled waveform generation
- Single synchronous 24 MHz clock domain
- Clock-enable based timing architecture
- FPGA-friendly implementation

AI: ChatGPT and Gemini

The project was developed with the heavy usage of AI tools. All the specification documents were created via talking sessions to chatGPT, most of them in voice chat on the mobile with follow ups on the keyboard.

Impementation, debugging and testing was done in VSCode with the free Gemini Extension. Later on, i switched the IDE to Antigravity and started paying for Gemini Access (Gemini Pro, Gemini Flash 3.5)

2. High-Level Architecture



3. Module Hierarchy

```
Synth
 ├── uart/ (Control Subsystem)
 │     └── Uart (Subsystem Wrapper)
 │           ├── UartRx
 │           ├── UartProtocolDecoder
 │           └── RegisterBank
 ├── common/ (Shared System Types)
 │     └── Types
 │
 ├── timing/ (System Control)
 │     └── TimingGenerator
 │
 ├── oscillator/ (Core Engine)
 │     └── Oscillator
 │           ├── Accumulator
 │           ├── Generators
 │           ├── Noise
 │           └── Mux
 │
 ├── envelope/ (Envelope Generator)
 │     └── EnvelopeGenerator
 │           ├── EnvelopeCtrl
 │           ├── EnvelopeAccumulator
 │           └── EnvelopeShaper
 │
 ├── mixing/ (Audio Processing)
 │     └── Attenuator (Volume Control)
 │
 └── output/ (Output Pipeline)
       ├── Decimator
       └── I2STransmitter
```

4. Master Clock

The complete design operates from a single synchronous master clock.

Parameter	Value
Master clock frequency	24 MHz

No internally-generated FPGA clocks shall be used.

All submodules shall operate synchronously from the 24 MHz master clock using clock-enable tick signals.

5. Timing Generators

The TimingGenerator module shall generate two independent clock-enable tick signals.

phaseTick

Parameter	Value
Frequency	480 kHz
Divider	24 MHz / 50
Purpose	Drive DDS phase accumulator

The phase accumulator and waveform generation logic shall update on this tick.

---

sampleTick

Parameter	Value
Frequency	48 kHz
Divider	24 MHz / 500
Purpose	Generate output audio samples

The decimator and output audio sample registers shall update on this tick.

---

6. Communication Protocol

The system is controlled via a standard UART interface. An external controller (such as a PC or Microcontroller) sends 3-byte packets to update the internal state of the synthesizer.

UART Configuration

Parameter	Value
Baud Rate	115,200
Data Bits	8
Parity	None
Stop Bits	1

Packet Format

The `UartProtocolDecoder` expects a 3-byte sequence for every command:

1. **Command Byte**: One byte for the command. (i.e. 0x01 for "write to register")
2. **Address Byte**: Specifies which register to write to.
3. **Data Byte**: The value to be written.

Command list

Right now there is only one command.

Command	Name	Adress Byte	Data Byte
`0x01`	`WriteRegister`	`From Register Map`	`1 Byte`

Register Map

Address	Register Name	Description	Width
`0x00`	`FREQ_LOW`	Frequency Word Bits [7:0]	8 bit
`0x01`	`FREQ_MID`	Frequency Word Bits [15:8]	8 bit
`0x02`	`FREQ_HIGH`	Frequency Word Bits [23:16]	8 bit
`0x03`	`WAVE_SEL`	0:Saw, 1:Square, 2:PWM, 3:Triangle, 4:Noise	3 bit
`0x04`	`PWM_WIDTH`	Duty cycle for PWM waveform	8 bit
`0x05`	`VOLUME`	Master output volume (Reserved)	8 bit
`0x40`	`ENV_CTRL`	Envelope Control: [0] Enable, [1] Gate, [2] Loop, [3] Ping-Pong, [4] Reverse, [6:5] Curve (00=Lin, 01=Exp, 10=Log, 11=S-Curve)	8 bit
`0x41`	`ENV_ATTACK`	Attack rate coefficient	8 bit
`0x42`	`ENV_DECAY`	Decay rate coefficient	8 bit
`0x43`	`ENV_SUSTAIN`	Sustain level (0 to 255)	8 bit
`0x44`	`ENV_RELEASE`	Release rate coefficient	8 bit
`0x45`	`ENV_SYNC_CTRL`	Sync Config: [0] Hard Sync, [1] Soft Sync, [2] MIDI Sync, [6:3] Clock Division Rate	8 bit
`0x46`	`ENV_PHASE_OFFSET`	Phase offset value (0 to 255)	8 bit

7. Oscillator Architecture

The oscillator shall use a classic DDS architecture.

Core

At every phaseTick:

```
phase := phase + freqWord
```

The phase accumulator shall wrap naturally on overflow.

Phase Accumulator

Parameter	Value
Width	24 bit
Type	Unsigned

Example:

```
val phase = Reg(UInt(24 bits))
```

Frequency Word

Parameter	Value
Width	24 bit
Type	Unsigned

The frequency word controls oscillator frequency.

> [!IMPORTANT] **Atomic Multi-Byte Update Protocol:** Since the 24-bit frequency word is spread across three 8-bit registers (`FREQ_LOW`, `FREQ_MID`, and `FREQ_HIGH`), updates are buffered atomically to prevent audio glitching:
>
> 1. Writing to `FREQ_LOW` (`0x00`) stages the lower 8 bits in a temporary shadow register.
> 2. Writing to `FREQ_MID` (`0x01`) stages the middle 8 bits in a temporary shadow register.
> 3. Writing to `FREQ_HIGH` (`0x02`) commits the entire 24-bit frequency word (`High ## MidShadow ## LowShadow`) simultaneously to the active synthesis registers in a single clock cycle.
>
> *Always write registers in order (`FREQ_LOW` → `FREQ_MID` → `FREQ_HIGH`) to ensure consistent updates.*

Frequency Calculation

The DDS frequency equation is:

```
f = freqWord × updateRate / 2^24
```

Where:

Parameter	Value
updateRate	480 kHz
phase width	24 bit

Frequency Resolution

The minimum frequency step is:

```
480000 / 16777216 ≈ 0.0286 Hz
```

---

8. Waveform Generators

The oscillator shall support the following waveforms.

Saw

Generated by mapping the upper phase bits to audio amplitude.

Example:

```
sample = phase[23:8]
```

Square

Generated using the phase accumulator MSB.

Example:

```
if phase[23] == 1:
    +MAX
else:
    -MAX
```

PWM

Generated using a comparator between phase and pulseWidth.

The 8-bit PWM width value shall be expanded internally before comparison with the 24-bit phase accumulator.

The expansion shall be implemented by shifting the PWM value 16 bits to the left to match 24 bits width.

Example:

```
if phase < pulseWidth:
    +MAX
else:
    -MAX
```

PWM Width

Parameter	Value
Width	8 bit
Type	Unsigned

Triangle

Generated using reflected phase arithmetic.

To generate the triangle wave, we utilize a "reflected phase" technique based on the 24-bit phase accumulator. The Most Significant Bit (MSB) of the phase acts as a direction indicator: during the first half-cycle (MSB=0), the lower 23 bits create a

linear rising ramp, whereas during the second half-cycle (MSB=1), those bits are bitwise inverted to produce a symmetrical falling ramp. This 23-bit result is then right-shifted by 7 bits to normalize it to a 16-bit range and cast to a signed integer (SInt), resulting in a full-swing bipolar waveform that transitions smoothly between peak amplitudes.

Noise

Noise generation shall use an LFSR-based pseudo-random generator.

Parameter	Value
Generator type	Fibonacci LFSR
Width	23 bit
Polynomial	x^23 + x^18 + 1
Feedback taps	bit 22 XOR bit 17
Update timing	`phaseTick`
Output type	16-bit signed
Reset seed	nonzero fixed value

An LFSR must never be initialized to zero, as it would stay stuck. We will plan to use a fixed non-zero seed.

The 16-bit signed audio is extracted just by taking the upper 16 bits of the LFSR.

---

9. Envelope Generator Architecture

The **Envelope Generator** is a control module designed to shape the volume (amplitude) or other modulation parameters of a sound over time. The general design principle is an ADSR engine.

When a key is pressed (Gate ON), the envelope rises to peak volume (Attack), decays slightly to a steady volume (Decay and Sustain), and then fades to silence when the key is released (Release).

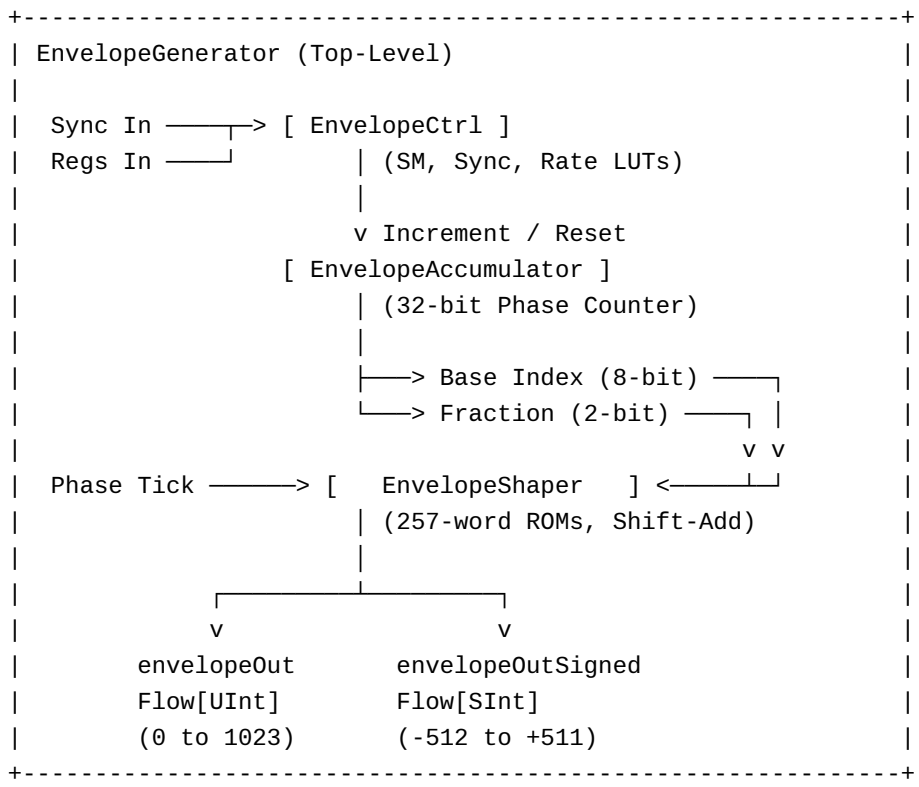
This module generates envelopes with a 10-bit resolution (0 to 1023) output value, which can be used as a volume (or other modulation) signal.

The entire module is designed with ASIC portability in mind, meaning it uses no specific hardware multipliers or memory blocks. Instead, it relies on compile-time Scala calculators to generate look-up curves in ROM, and performs most intermediate steps using bit-shifts and additions.

9.1 EnvelopeGenerator: Top-Level Wrapper

The top-level `EnvelopeGenerator` module integrates the submodules and registers them to the system communication and audio pipelines.

System Diagram



Module Interface (I/O Ports)

The top-level `EnvelopeGenerator` operates directly on the 24 MHz main system clock and exposes the following SpinalHDL hardware IO bundle:

```
val io = new Bundle {
  // Inputs
  val phaseTick = in Bool()                // Heartbeat tick synced with 480 kHz
sample rate
  val syncIn    = in Bool()                // External trigger for Hard or Soft Sync
  val midiClock = in Bool()                // External MIDI clock tick (24 PPQN pulse)
  val config    = in(EnvelopeConfig())     // Packaged register configurations

  // Outputs
  val envelopeOut       = master(Flow(UInt(10 bits))) // Unipolar output (0 to 1023)
  val envelopeOutSigned = master(Flow(SInt(10 bits))) // Bipolar output (-512 to +511)
}
```

- **Unipolar Output (envelopeOut):** Emits unsigned 10-bit values (0 to 1023) for standard amplitude scaling or unipolar modulation. The flow's `valid` signal is synchronized to `phaseTick` (480 kHz heartbeat).
- **Bipolar Output (envelopeOutSigned):** Emits signed 10-bit values (-512 to +511) for ring modulation, phase modulation, or center-zero pitch modulations. The flow's `valid` signal is synchronized to `phaseTick` (480 kHz heartbeat).

The EnvelopeConfig Bundle

Following the consistent design patterns of the synthesizer's components, the parameter configuration is packaged into a unified Scala bundle under the `synth.common` package:

```
case class EnvelopeConfig() extends Bundle {
  val ctrl        = Bits(8 bits)
  val attack      = UInt(8 bits)
  val decay       = UInt(8 bits)
  val sustain     = UInt(8 bits)
  val release     = UInt(8 bits)
  val syncCtrl    = Bits(8 bits)
  val phaseOffset = UInt(8 bits)
}
```

Register Map

The following registers are mapped into the `spinalSynth` SPI/UART register bus to control the Generator parameters:

Register Address (Hex)	Register Name	Bit Width	Description
`0x40`	`ENV_CTRL`	8 bits	Control bits: `[0]` Enable, `[1]` Gate, `[2]` Loop (LFO), `[3]` Ping-Pong (Reserved Placeholder), `[4]` Reverse (Reserved Placeholder), `[6:5]` Curve Model (`00`=Lin, `01`=Exp, `10`=Log, `11`=S-Curve)
`0x41`	`ENV_ATTACK`	8 bits	Attack rate coefficient (speed of phase accumulator in Attack)
`0x42`	`ENV_DECAY`	8 bits	Decay rate coefficient
`0x43`	`ENV_SUSTAIN`	8 bits	Sustain Level (0 to 255, scaled to 10-bit range internally)
`0x44`	`ENV_RELEASE`	8 bits	Release rate coefficient
`0x45`	`ENV_SYNC_CTRL`	8 bits	Sync config: `[0]` Hard Sync Enable, `[1]` Soft Sync Enable, `[2]` MIDI Sync Enable, `[6:3]` Clock Division Rate
`0x46`	`ENV_PHASE_OFFSET`	8 bits	Phase offset value (0 to 255 representing 0 to 360 degrees)

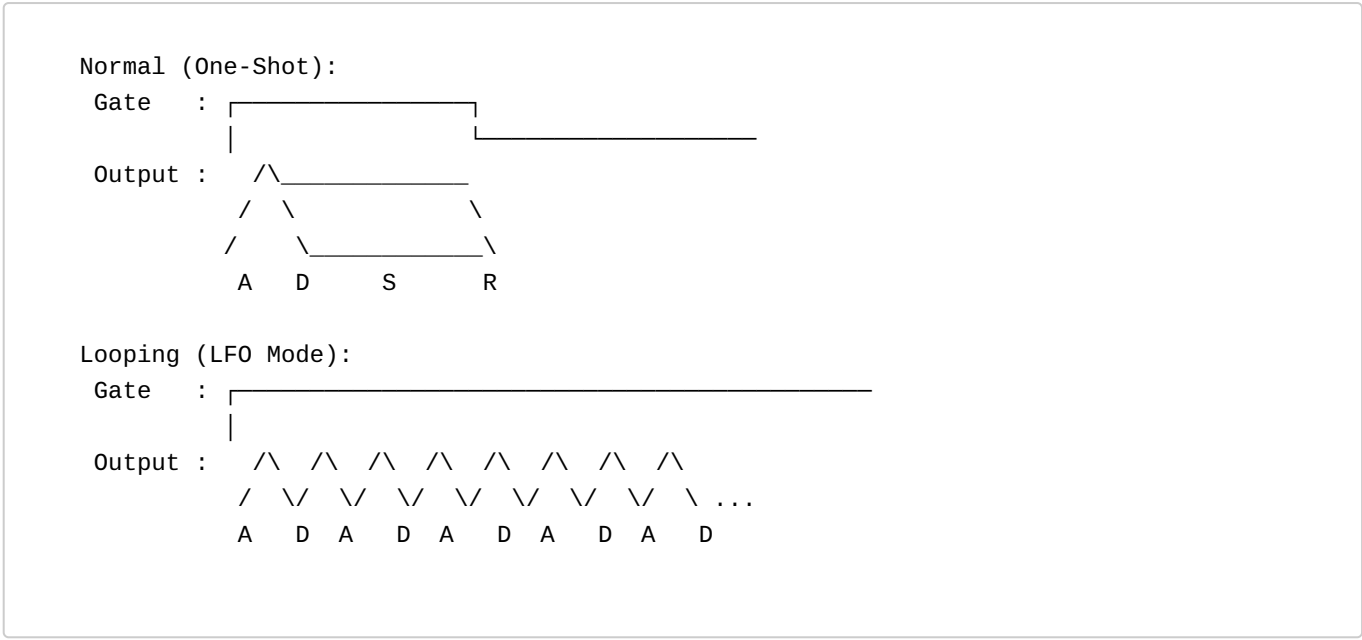
9.2 EnvelopeCtrl

`EnvelopeCtrl` is the state machine and synchronization module that determines the active phase increment values and the play direction.

9.2.1 ADSR & Playback Modes

There are different modes for the ADSR playback envelopes and shapes:

- **Normal (One-Shot):** Triggers on Gate ON, transitions from Attack to Decay to Sustain, and goes to Release on Gate OFF.
- **Looping (LFO Mode):** The envelope automatically loops back to the start of the Attack phase once the Decay phase finishes.
- **Reverse Mode (Reserved Placeholder):** Retained in register map; not implemented in active RTL logic.
- **Ping-Pong Mode (Reserved Placeholder):** Retained in register map; not implemented in active RTL logic.



9.2.2 Sync and Phase

- **Hard Sync:** An external sync trigger instantly resets the current envelope phase accumulator back to 0 (start of Attack) and restarts the state machine.
- **Soft Sync (Reserved Placeholder):** Bypassed in current active RTL.
- **MIDI Sync & Clock Division (Reserved Placeholder):** Bypassed in current active RTL.

9.2.3 AD(S)R Lengths: Time Duration Mapping

In synthesizer design, how parameter values map to actual time durations directly determines the musical feel of the instrument.

Why Linear Mapping Fails

If we map the 8-bit parameters (0 to 255) of the Attack, Decay and Release registers to time durations linearly, we encounter severe playing issues:

- **Linear Time Mapping:** If time increases linearly up to 30.0 seconds, the first step is already 117 milliseconds. This completely wipes out snappy, high-energy percussion attacks (which require precise control between 1 ms and 50 ms).
- **Linear Increment Mapping:** Mapping step size (increment) linearly creates a hyperbola where most of the range is crammed into tiny millisecond adjustments at the fast end, making it practically impossible to select slow durations with any precision.

The Logarithmic Solution

To match human hearing perception, we use a **logarithmic time mapping** (exponential increments). This splits the 8-bit parameter range into three playable musical zones:

- **Register Values 0 to 100:** Snappy transients (0.5 ms to 200 ms) with sub-millisecond precision.
- **Register Values 100 to 200:** Medium decay and release controls (200 ms to 3.0 seconds).
- **Register Values 200 to 255:** Very slow, evolving ambient sweeps (3.0 seconds to 30.0 seconds).

Hardware Implementation: Pre-Calculated ROM

Calculating logarithmic curves or exponential step values at runtime is expensive in ASIC silicon, requiring division blocks and exponential math units.

To maintain ASIC portability, we pre-calculate the 256 increment step values in Scala at compile-time. When a parameter register (Attack, Decay, or Release) is written, the system simply uses the 8-bit value to index a static lookup ROM (256 words x 22-bit width) to retrieve the accumulator step size instantly.

```
System Specifications:
  mainClock        = 24 MHz system clock
  T_min            = 0.5 ms (0.0005 seconds)
  T_max            = 30.0 seconds
  Accumulator Width = 32 bits (10 bits integer + 22 bits fraction)
  Increment Width   = 22 bits

Mathematical Model:
  T(P)             = T_min * (T_max / T_min) ^ (P / 255)
  increment(P)     = 2^32 / (T(P) * 24,000,000)
```

9.3 EnvelopeAccumulator

The `EnvelopeAccumulator` acts as the time-tracking motor of the envelope generator, capable of counting in both directions (forward and reverse) depending on the active stage.

How the Accumulator works

The accumulator is a 32-bit register. On every master clock cycle (24 MHz), the accumulator adds (or subtracts) the active 22-bit phase increment value:

- **Attack (Stage 1)**: Counts UP (`accumDir = 0`). `segmentDone` triggers when it overflows past `1023` (representing index 255).
- **Decay (Stage 2)**: Counts DOWN (`accumDir = 1`). `segmentDone` triggers when the integer `baseIndex` matches or crosses below `sustainLevel` (using an 8-bit hardware comparator).
- **Sustain (Stage 3)**: Paused (`runAccum = 0`), naturally holding the output stable at `sustainLevel` without any pipeline registers.
- **Release (Stage 4)**: Counts DOWN (`accumDir = 1`). `segmentDone` triggers when it underflows (representing `baseIndex` reaching `0`).

Clock and Frequency Boundaries

At the 24 MHz main clock rate with a 32-bit accumulator and 22-bit phase increment, the exact operational limits are calculated as follows:

Target Speed Limit	Time Duration	Active Clock Cycles	Calculated Increment (Decimal)	Increment (Hexadecimal)
Maximum Speed (T_min)	0.5 milliseconds	12,000 cycles	**357,914**	0x05761A
Minimum Speed (T_max)	30.0 seconds	720,000,000 cycles	**6**	0x000006

Hardware Phase Specifications

- **Accumulator Size:** Uses a 32-bit phase accumulator (10 bits integer + 22 bits fraction).
- **Segment Limits:** Evaluated dynamically based on FSM state (`overflow` for Attack, `baseIndex <= sustainLevel` for Decay, `underflow` for Release).
- **Output Splitting:** Splits the upper 10 integer bits of the active 32-bit phase (bits 31 to 22) into two fields to drive the waveshaper:
  - **Base Index:** The higher 8 bits of the integer part (bits 31 to 24), representing the active step index (0 to 255).
  - **Fractional Part:** The lower 2 bits of the integer part (bits 23 to 22), representing the interpolation fraction (0 to 3).

9.4 EnvelopeShaper

The `EnvelopeShaper` is the output stage of the envelope generator.

It takes the raw, linear ramp outputs from the accumulator and transforms them into customized, musically natural curves. It reads two consecutive points from a 257-entry curve ROM (Lin, Exp, Log, S-Curve) based on the 8-bit Base Index, performs linear interpolation in pure multiplierless combinational logic using the 2-bit fraction, and outputs unipolar/bipolar audio-rate flows.

9.4.1 ROM Lookup tables

The Base Index (upper 8 bits) addresses lookup curves from pre-calculated 257-word ROMs (257 x 8 bits) using these profiles:

Curve Model	Description	Primary Audio Application
Linear (Lin)	Perfectly straight transition lines.	LFO sweeps, pitch modulation, physical modeling.
Exponential (Exp)	Accelerating curve start, mimicking natural capacitor discharge.	Snappy percussion envelopes, natural string plucks.
Logarithmic (Log)	Rapid initial rise followed by gradual flattening.	High-energy attack dynamics, volume compensation.
S-Curve (Sigmoid)	Smooth cosine-like ease-in and ease-out transitions.	Smooth organic sweeps, cinematic pads, crossfading.

The 257-Entry ROM Boundary Safeguard

To calculate Y1 = LUT[x+1] when the base index is at its boundary (x = 255) without conditional bounds checking or wrapping, the curve ROM is constructed with **257 entries** (indices 0 to 256). For x = 255, LUT[x+1] safely returns LUT[256], containing the true terminal amplitude value.

9.4.2 Hybrid 8+2 Bit Interpolation Math

Using the splits from the 10 bits accumulator output:

- ☑ and Y1 = LUT[x+1].
- The 2-bit fraction f represents step fractions {0, 1/4, 2/4, 3/4}.
- **Interpolation**: Evaluates `Y = Y0 + (f / 4) * (Y1 - Y0)`.
- **Reverse Gating**: When counting backwards (`accumDir = 1` during Decay and Release), the 2-bit fractional index is mirrored combinationally: `fractionAdjusted = accumDir ? (3 - fraction) : fraction` This guarantees that interpolation sweeps smoothly and linearly downward in both directions.

9.4.3 Multiplierless Shift-Add Implementation

The fractional calculation is implemented in pure combinational shift-add logic:

Fractional Bits (f_adjusted)	Fraction Value	Hardware Shift-Add Expression
`00`	0.00	Y0
`01`	0.25	Y0 + (delta Y >> 2)
`10`	0.50	Y0 + (delta Y >> 1)
`11`	0.75	Y0 + (delta Y >> 1) + (delta Y >> 2)

Since the accumulator physically halts at `sustainLevel` during Sustain and counts backwards naturally during Decay and Release, **no multipliers or sustain delay pipelines are required**, making the output stage extremely area-efficient.

---

10. Attenuation & Volume Control

Volume level control is performed at the oversampled 480 kHz rate prior to decimation by the `Attenuator` module. To maximize reusable modularity (e.g., interfacing with an 8-bit manual volume register or a 10-bit dynamic envelope generator output), the `Attenuator` is designed as a compile-time parameterized component:

- **Compile-Time Parameter:** `volumeWidth: Int = 8`
- **Volume Input Port:** `io.volume: UInt(volumeWidth bits)`
- **Mathematical Operation:**

  ```
  scaledSample = (sampleIn * volumeSigned) >> volumeWidth
  ```

  This is implemented efficiently in hardware using a single signed multiplier and bitwise shift scaling, leaving the original sample rate and 16-bit audio resolution unaltered.

---

11. Oversampling and Decimation

Oversampling Strategy

The oscillator shall internally operate at:

```
480 kHz
```

while the final audio output sample rate shall be:

```
48 kHz
```

This creates an oversampling ratio of:

```
10×
```

Decimation Strategy

The implementation shall use simple zero-order decimation.

Every 10th sample shall be captured as the output audio sample.

No interpolation or low-pass filtering shall initially be used.

Example:

```
if(sampleTick) {
    audioSample := oscSample
}
```

12. Audio Sample Format

Parameter	Value
Audio width	16 bit
Sample format	Signed
Sample rate	48 kHz

Example:

```
val sample = SInt(16 bits)
```

The oscillator is currently mono internally.

The mono signal shall be duplicated to both stereo output channels.

Example:

```
leftSample  = sample
rightSample = sample
```

13. I²S Output Interface

The output interface shall use the I²S protocol.

I²S Timing Architecture

The I²S transmitter shall operate directly from the 24 MHz master clock. The transmitter shall use a cycle-timed state machine architecture.

I²S Bit Timing

The required I²S bit clock frequency BCLK is:

```
48,000 × 2 × 16 = 1.536 MHz
```

The relationship to the 24 MHz master clock is:

```
24 MHz / 1.536 MHz = 15.625
```

Therefore no integer divider exists.

The serializer shall therefore alternate between:

- 15 master-clock cycles
- 16 master-clock cycles

between serialized bit transfers.

I²S Timing Subpattern

The serializer shall use the following repeating 8-step timing subpattern:

```
16,16,15,16,16,15,16,15
```

This subpattern contains:

Interval	Count
16-cycle intervals	5
15-cycle intervals	3

Total clocks:

```
16+16+15+16+16+15+16+15 = 125
```

Average clocks per bit:

```
125 / 8 = 15.625
```

This exactly matches the required average I²S bit timing.

Relationship To LRCLK Period

One stereo I²S frame contains:

```
32 serial bits
```

because:

- 16 left-channel bits
- 16 right-channel bits

Since:

```
32 = 4 × 8
```

the 8-step timing subpattern repeats exactly four times during one complete stereo frame.

Full frame timing:

```
[16,16,15,16,16,15,16,15] × 4
```

Total master-clock cycles per stereo frame:

```
4 × 125 = 500
```

Stereo frame rate:

```
24 MHz / 500 = 48 kHz
```

This produces the exact required audio sample rate.

I^2S Timing State Machine

The serializer shall internally contain:

Register	Purpose
cycleCounter	Current interval countdown
patternIndex	Selects 15/16-cycle interval
bitCounter	Counts serialized bits
shiftRegister	Serialized audio data

The pattern index shall cycle continuously:

```
0 → 1 → 2 → ... → 7 → 0
```

The bit counter shall cycle:

```
0 → 1 → 2 → ... → 31 → 0
```

The bit counter determines:

- LRCLK state

- stereo frame boundaries
- sample reload timing

I²S Audio Format

Parameter	Value
Channels	2
Audio width	16 bit
Sample rate	48 kHz
Bit clock	1.536 MHz

I²S Signals

Signal	Description
i2s_bclk	Bit clock
i2s_lrclk	Left/right word select
i2s_sdata	Serial audio data

Serializer Behavior

The I²S serializer shall:

- shift audio data
- serialize stereo audio samples
- generate LRCLK framing
- output signed 16-bit audio samples

The exact serializer state machine behavior is not yet specified.

---

14. Numeric Formats

Signal	Type
phase	UInt(24 bits)
freqWord	UInt(24 bits)
pulseWidth	UInt(8 bits)
audioSample	SInt(16 bits)
volume	UInt(volumeWidth bits)

The design shall use fixed-point arithmetic throughout.

Grouped Bundles & Flow Interfaces

Bundle	Subfields	Type
RegisterWrite	`address` `data`	UInt(8 bits) Bits(8 bits)

OscillatorConfig	`freqWord`
	`waveSelect`
	`pwmWidth`
	`volume`
		UInt(24 bits)
		UInt(3 bits)
		UInt(8 bits)
		UInt(8 bits)

Waveforms	`saw`
	`square`
	`pwm`
	`tri`
		SInt(16 bits)
		SInt(16 bits)
		SInt(16 bits)
		SInt(16 bits)

15. Confirmed System Parameters

Parameter	Value
HDL	SpinalHDL
Master clock	24 MHz
DDS phase width	24 bit
DDS update rate	480 kHz
Audio sample rate	48 kHz
Audio width	16 bit signed
I^2S output	Stereo
I^2S bit clock	1.536 MHz
Oversampling ratio	10×
Decimation method	Every 10th sample
Arithmetic	Fixed-point
Waveforms	Saw, Square, PWM, Triangle, Noise
Clocking strategy	Single synchronous clock domain